

MI VDF API

Version 2.09

© 2022 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

REVISION HISTORY

Revision No.	Description	Date
2.03	Initial release	11/12/2018
2.04	Document refinement	04/12//2019
2.05	- Add Work Flow - Fix MD param info	10/12/2020
2.06	- Fix VG param info. - Add description of new structure.	11/02/2020
2.07	- Add Flowchart Title - Add MI_MD_ResultSize_t struct - Fix variables and funcs hyperlinks - Add MI_OD_Result_t struct member - Add proc debug info	11/10/2021
2.08	Change 16RstBufSize to u32RstBufSize	03/28/2022
2.09	Add 64bit physical address of MD/OD	04/12/2022

TABLE OF CONTENTS

REVISION HISTORY.....	i
TABLE OF CONTENTS.....	ii
1. 概述.....	1
1.1. 模块说明.....	1
1.2. 流程框图.....	1
1.3. 关键字说明.....	3
2. API 参考.....	4
2.1. API 列表.....	4
2.2. API 说明.....	4
2.2.1 MI_VDF_Init.....	4
2.2.2 MI_VDF_Uninit.....	5
2.2.3 MI_VDF_CreateChn.....	5
2.2.4 MI_VDF_DestroyChn.....	6
2.2.5 MI_VDF_SetChnAttr.....	7
2.2.6 MI_VDF_GetChnAttr.....	8
2.2.7 MI_VDF_EnableSubWindow.....	8
2.2.8 MI_VDF_Run.....	9
2.2.9 MI_VDF_Stop.....	10
2.2.10 MI_VDF_GetResult.....	10
2.2.11 MI_VDF_PutResult.....	11
2.2.12 MI_VDF_GetLibVersion.....	12
2.2.13 MI_VDF_GetDebugInfo.....	12
3. 数据类型.....	14
3.1. 结构体列表.....	14
3.2. VDF 结构体说明.....	14
3.2.1 MI_VDF_WorkMode_e.....	14
3.2.2 MI_VDF_Color_e.....	15
3.2.3 MI_VDF_ODWindow_e.....	15
3.2.4 MI_MD_Result_t.....	16
3.2.5 MI_OD_Result_t.....	17
3.2.6 MI_VG_Result_t.....	17
3.2.7 MI_VDF_Result_t.....	18
3.2.8 MI_VDF_MdAttr_t.....	19
3.2.9 MI_VDF_OdAttr_t.....	19
3.2.10 MI_VDF_VgAttr_t.....	20
3.2.11 MI_VDF_ChnAttr_t.....	21
3.2.12 MDRST_STATUS_t.....	22
3.2.13 MI_MD_ResultSize_t.....	22
3.3. MD 结构体列表.....	23
3.4. MD 结构体说明.....	23
3.4.1 MDMB_MODE_e.....	23
3.4.2 MDSAD_OUT_CTRL_e.....	24

3.4.3	MDALG_MODE_e.....	24
3.4.4	MDCCL_ctrl_t.....	25
3.4.5	MDPreproc_ctrl_t.....	25
3.4.6	MDBlock_info_t.....	26
3.4.7	MDPoint_t.....	26
3.4.8	MDROI_t.....	27
3.4.9	MDSAD_DATA_t.....	27
3.4.10	MDOBJ_t.....	28
3.4.11	MDOBJ_DATA_t.....	28
3.4.12	MI_MD_IMG_t.....	29
3.4.13	MI_MD_IMG64_t.....	29
3.4.14	MI_MD_static_param_t.....	30
3.4.15	MI_MD_param_t.....	30
3.5.	OD 结构体列表.....	31
3.6.	OD 结构体说明.....	32
3.6.1	MI_OD_WIN_STATE.....	32
3.6.2	ODColor_e.....	32
3.6.3	ODWindow_e.....	33
3.6.4	ODPoint_t.....	33
3.6.5	ODROI_t.....	34
3.6.6	MI_OD_IMG_t.....	34
3.6.7	MI_OD_IMG64_t.....	35
3.6.8	MI_OD_static_param_t.....	35
3.6.9	MI_OD_param_t.....	36
3.7.	VG 结构体列表.....	37
3.8.	VG 结构体说明.....	37
3.8.1	VgFunction.....	37
3.8.2	VgRegion_Dir.....	38
3.8.3	VgSize_Sensitively.....	38
3.8.4	VgDirection_State.....	39
3.8.5	MI_VG_Point_t.....	39
3.8.6	MI_VgLine_t.....	40
3.8.7	MI_VgRegion_t.....	40
3.8.8	MI_VgSet_t.....	41
3.8.9	MI_VgResult_t.....	42
3.8.10	MI_VgBoundingBox_t.....	43
3.8.11	MI_VgDetectThd.....	43
4.	错误码.....	44
5.	PROCFS INTRODUCTION.....	46
5.1.	cat.....	46
5.2.	echo.....	54

1. 概述

1.1. 模块说明

VDF 实现 MD, OD, VG 视频通道的初始化, 通道管理, 视频检测结果的管理和通道销毁等功能。

1.2. 流程框图

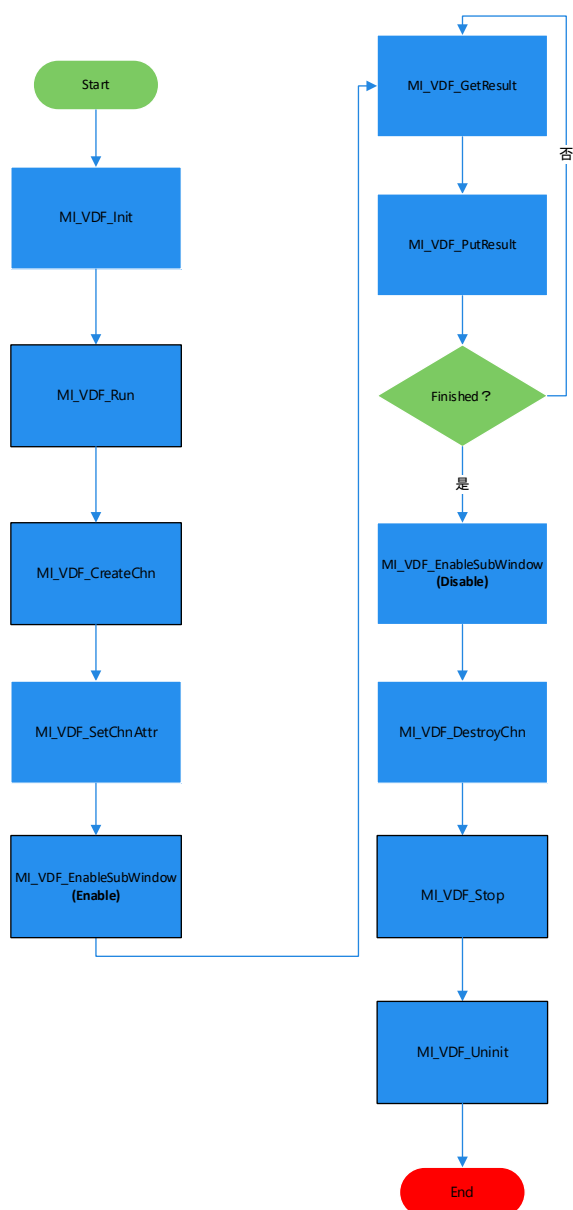


图 1-1 一路 VDF 通道的函数调用流程

上图是开启一路 VDF 的调用流程，但是 VDF 可以支持多路通道，每个通道都可以设置 MD/OD/VG 其中一种工作模式。

每一路 VDF 通道，都要独立调用

[MI_VDF_CreateChn](#)、[MI_VDF_SetChnAttr](#)、[MI_VDF_EnableSubWindow](#)、[MI_VDF_GetResult](#)、[MI_VDF_PutResult](#)、[MI_VDF_DestroyChn](#) 等 API 做通道控制，通道属性，销毁等动作。

如下图所示，开启了 4 个 VDF 通道，其中通道 0 和通道 1 设置为 MD 模式，通道 2 和通道 3 设置为 OD 模式。

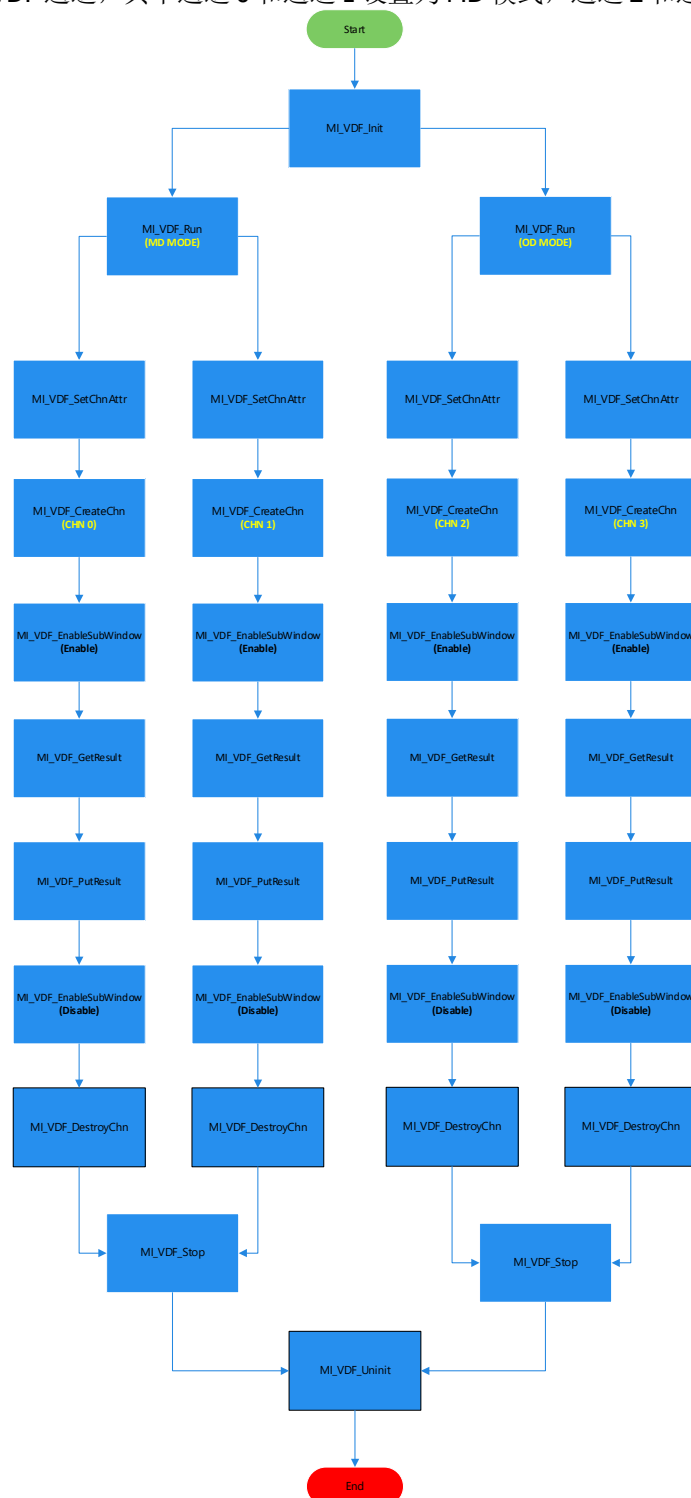


图 1-2 4 路 VDF 通道的函数调用流程

1.3. 关键字说明

- MD: 移动侦测
- OD: 遮挡侦测
- VG: 虚拟围栏

2. API 参考

2.1. API 列表

API 名	功能
MI_VDF_Init	初始化 MI_VDF 模块
MI_VDF_Uninit	析构 MI_VDF 模块
MI_VDF_CreateChn	创建视频侦测(MD/OD/VG)通道
MI_VDF_DestroyChn	销毁已创建的视频侦测（MD/OD/VG）通道
MI_VDF_SetChnAttr	设置视频侦测（MD/OD/VG）通道属性
MI_VDF_GetChnAttr	获取视频侦测（MD/OD/VG）通道属性
MI_VDF_EnableSubWindow	使能 VDF 视频侦测（MD/OD/VG）通道子窗口
MI_VDF_Run	开始运行视频侦测（MD/OD/VG）
MI_VDF_Stop	停止运行视频侦测（MD/OD/VG）
MI_VDF_GetResult	获取视频侦测（MD/OD/VG）结果
MI_VDF_PutResult	释放视频侦测（MD/OD/VG）结果
MI_VDF_GetLibVersion	获取 VDF 指定通道（MD/OD/VG）的版本号
MI_VDF_GetDebugInfo	获取 VDF 指定通道的 Debuginfo(only VG 支持)

2.2. API 说明

2.2.1 MI_VDF_Init

- 功能
 - 初始化 VDF 模块。
- 语法


```
MI_S32 MI_VDF_Init(void);
```
- 形参
 - 无
- 返回值

返回值

{

0 成功。

非 0 失败，参照[错误码](#)。
- 依赖
 - 头文件：mi_sys.h、mi_md.h、mi_od.h、mi_vg.h、mi_vdf.h
 - 库文件：libOD_LINUX.a、libMD_LINUX.a、libVG_LINUX.a、libmi_vdf.a

- ※ 注意
 - [MI_VDF_Init](#)需要在调用 [MI_SYS_Init](#)之后调用。
 - 不可以重复调用 [MI_VDF_Init](#)。

- 举例
无

- 相关主题
[MI_VDF_Uninit](#)

2.2.2 MI_VDF_Uninit

- 功能
析构 MI_VDF 系统，调用 MI_VDF_Uninit 之前，需要确保已创建的 VDF 通道都已经被 disable，所有通道的视频检测结果都被释放。
- 语法
`MI_S32 MI_VDF_Uninit (void);`
- 形参
无
- 返回值
返回值 $\left\{ \begin{array}{l} 0 \text{ 成功。} \\ \text{非 } 0 \text{ 失败，参照} \text{ [错误码](#)。} \end{array} \right.$
- 依赖
 - 头文件：mi_sys.h、mi_md.h、mi_od.h、mi_vg.h、mi_vdf.h
 - 库文件：libOD_LINUX.a、libMD_LINUX.a、libVG_LINUX.a、libmi_vdf.a
- ※ 注意
 - [MI_VDF_Uninit](#)调用前需要确保所有创建的 VDF 通道都处于 disable 状态。
 - [MI_VDF_Uninit](#)调用前需要确保所有创建的 VDF 通道的结果都释放。
- 举例
参见 [MI_VDF_Init](#) 举例。
- 相关主题
无

2.2.3 MI_VDF_CreateChn

- 功能
创建视频侦测(MD/OD/VG)通道。
- 语法
`MI_S32 MI_VDF_CreateChn(MI_VDF_CHANNEL VdfChn, const MI\_VDF\_ChnAttr\_t* pstAttr);`

➤ 形参

参数名称	参数含义	输入/输出
VdfChn	指定视频侦测通道号。	输入
pstAttr	设置视频通道属性值。	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys.h、mi_md.h、mi_od.h、mi_vg.h、mi_vdf.h
- 库文件：libOD_LINUX.a、libMD_LINUX.a、libVG_LINUX.a、libmi_vdf.a

※ 注意

- VdfChn 不能重复指定。
- VdfChn 的有效值范围：0 ≤ VdfCh < MI_VDF_CHANNEL_MAX，MI_VDF_CHANNEL_MAX 定义在 mi_vdf_datatype.h。

➤ 举例

无。

➤ 相关主题

[MI_VDF_DestroyChn](#)

2.2.4 MI_VDF_DestroyChn

➤ 功能

销毁已创建的视频侦测通道。

➤ 语法

```
MI_S32 MI_VDF_DestroyChn(MI_VDF_CHANNEL VdfChn);
```

➤ 形参

参数名称	描述	输入/输出
VdfChn	已经创建的视频侦测通道号。	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys.h、mi_md.h、mi_od.h、mi_vg.h、mi_vdf.h
- 库文件：libOD_LINUX.a、libMD_LINUX.a、libVG_LINUX.a、libmi_vdf.a

※ 注意

- VdfChn 必须是已经创建的视频侦测通道。
- VdfChn 的有效值范围：0 ≤ VdfCh < MI_VDF_CHANNEL_MAX，MI_VDF_CHANNEL_MAX 定义在 mi_vdf_datatype.h。

- 举例
无。
- 相关主题
无。

2.2.5 MI_VDF_SetChnAttr

- 功能
设置视频侦测通道的属性。
- 语法
MI_S32 MI_VDF_SetChnAttr(MI_VDF_CHANNEL VdfChn, const [MI_VDF_ChnAttr_t](#)* pstAttr);

- 形参

参数名称	描述	输入/输出
VdfChn	已经创建的视频侦测通道号。	输入
pstAttr	源端口配置信息数据结构指针。	输入

- 返回值
返回值 $\begin{cases} 0 & \text{成功。} \\ \text{非 } 0 & \text{失败，参照[错误码](#)。} \end{cases}$

- 依赖
 - 头文件：mi_sys.h、mi_md.h、mi_od.h、mi_vg.h、mi_vdf.h
 - 库文件：libOD_LINUX.a、libMD_LINUX.a、libVG_LINUX.a、libmi_vdf.a

- ※ 注意
 - VdfChn 必须是已经创建的视频侦测通道。
 - 该接口指定设置视频侦测通道的动态属性值，即 [MI_VDF_ChnAttr_t](#) 中的 [stMdAttr.stMdDynamicParamsIn](#) / [stMdAttr.stOdDynamicParamsIn](#) / [stMdAttr.stVgAttr](#) 部分。

- 举例
无
- 相关主题
无

2.2.6 MI_VDF_GetChnAttr

- 功能
获取视频侦测通道属性。
- 语法
MI_S32 MI_VDF_GetChnAttr(MI_VDF_CHANNEL VdfChn, [MI_VDF_ChnAttr_t](#)* pstAttr);
- 形参

参数名称	描述	输入/输出
VdfChn	已经创建的视频侦测通道号。	输入
pstAttr	用来保存返回的视频侦测通道属性值	输出

➤ 返回值

返回值 { 0 成功。
非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys.h、mi_md.h、mi_od.h、mi_vg.h、mi_vdf.h
- 库文件：libOD_LINUX.a、libMD_LINUX.a、libVG_LINUX.a、libmi_vdf.a

※ 注意

无

➤ 举例

无

➤ 相关主题

无

2.2.7 MI_VDF_EnableSubWindow

➤ 功能

使能/关闭指定的视频侦测通道子窗口。

➤ 语法

```
MI_S32 MI_VDF_EnableSubWindow(MI_VDF_CHANNEL VdfChn, MI_U8 u8Col, MI_U8 u8Row, MI_U8 u8Enable);
```

➤ 形参

参数名称	描述	输入/输出
VdfChn	已经创建的视频侦测通道号。	输入
u8Col	子窗口的行地址(保留参数，暂不使用)	输入
u8Row	子窗口的列地址(保留参数，暂不使用)	输入
u8Enable	子窗口的使能控制标志	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：Mi_sys.h、mi_md.h、mi_od.h、mi_vg.h、mi_vdf.h
- 库文件：libOD_LINUX.a、libMD_LINUX.a、libVG_LINUX.a、libmi_vdf.a

※ 注意

- u8Col, u8Row 为 API 接口向上兼容而保留，暂无实际作用，可以直接赋值 0。

- 工作模式目前支持三种：MD，OD，VG。
- [MI_VDF_Run/MI_VDF_Stop](#) 函数作用的是整个工作模式（MD/OD/VG）下的所有视频侦测通道；而 [MI_VDF_EnableSubWindow](#) 则是作用于单个视频侦测通道。对于一个 VdfChn 来说，只有 [MI_VDF_Run](#) 和 [MI_VDF_EnableSubWindow](#) 都开启，才会开始运行。只要调用 [MI_VDF_Stop](#) 或者调用 [MI_VDF_EnableSubWindow](#) 的时候 u8Enable 设置成 false，都会停止。

- 举例
无。
- 相关主题
无。

2.2.8 MI_VDF_Run

- 功能
开启指定的视频侦测工作模式（MD/OD/VG）。
- 语法
MI_S32 MI_VDF_Run([MI_VDF_WorkMode_e](#) enWorkMode);

- 形参

参数名称	描述	输入/输出
enWorkMode	视频侦测（MD/OD/VG）工作模式。	输入

- 返回值

返回值	$\left\{ \begin{array}{l} 0 \text{ 成功。} \\ \text{非 } 0 \text{ 失败，参照错误码。} \end{array} \right.$
-----	---

- 依赖
 - 头文件：mi_sys.h、mi_md.h、mi_od.h、mi_vg.h、mi_vdf.h
 - 库文件：libOD_LINUX.a、libMD_LINUX.a、libVG_LINUX.a、libmi_vdf.a

- ※ 注意
 - 工作模式目前支持三种：MD，OD，VG。
 - [MI_VDF_Run/MI_VDF_Stop](#) 函数作用于整个工作模式（MD/OD/VG）下的所有视频侦测通道。

- 举例
无。
- 相关主题
无。

2.2.9 MI_VDF_Stop

- 功能
停止视频侦测（MD/OD/VG）工作模式。
- 语法
MI_S32 MI_VDF_Stop([MI_VDF_WorkMode_e](#) enWorkMode);

➤ 形参

参数名称	描述	输入/输出
enWorkMode	视频侦测（MD/OD/VG）工作模式。	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys.h、mi_md.h、mi_od.h、mi_vg.h、mi_vdf.h
- 库文件：libOD_LINUX.a、libMD_LINUX.a、libVG_LINUX.a、libmi_vdf.a

※ 注意

- 工作模式目前支持三种：MD，OD，VG。
- [MI_VDF_Run/MI_VDF_Stop](#) 函数作用于整个工作模式（MD/OD/VG）下的所有视频侦测通道。

➤ 举例

无。

➤ 相关主题

无。

2.2.10 MI_VDF_GetResult

➤ 功能

获取指定视频侦测通道的检测结果。

➤ 语法

```
MI_S32 MI_VDF_GetResult(MI_VDF_CHANNEL VdfChn, MI_VDF_Result_t* pstVdfResult, MI_S32 s32MilliSec);
```

➤ 形参

参数名称	描述	输入/输出
VdfChn	已经创建的视频侦测通道号。	输入
pstVdfResult	用来保存返回的视频侦测结果	输出
s32MilliSec	输入时间参数(保留参数，暂不使用)	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys.h、mi_md.h、mi_od.h、mi_vg.h、mi_vdf.h
- 库文件：libOD_LINUX.a、libMD_LINUX.a、libVG_LINUX.a、libmi_vdf.a

※ 注意

- VdfChn 必须是已经创建的视频侦测通道。
- s32MilliSec 为 API 接口向上兼容而保留，暂无实际作用，可以直接赋值 0。
- 用于保存返回结果的指针不能为空。

➤ 举例

无

➤ 相关主题

无。

2.2.11 MI_VDF_PutResult

➤ 功能

释放指定视频侦测通道的检测结果。

➤ 语法

MI_S32 MI_VDF_PutResult(MI_VDF_CHANNEL VdfChn, [MI_VDF_Result_t](#)* pstVdfResult);

➤ 形参

参数名称	描述	输入/输出
VdfChn	已经创建的视频侦测通道号。	输入
pstVdfResult	指定需要释放视频通道的结果参数	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys.h、mi_md.h、mi_od.h、mi_vg.h、mi_vdf.h
- 库文件：libOD_LINUX.a、libMD_LINUX.a、libVG_LINUX.a、libmi_vdf.a

※ 注意

- VdfChn 必须是已经创建的视频侦测通道。

➤ 举例

无。

➤ 相关主题

无。

2.2.12 MI_VDF_GetLibVersion

➤ 功能

获取 MD/OD/VG 库版本号。

➤ 语法

MI_S32 MI_VDF_GetLibVersion(MI_VDF_CHANNEL VdfChn, MI_U32* u32VDFVersion);

➤ 形参

参数名称	描述	输入/输出
VdfChn	已经创建的视频侦测通道号。	输入
u32VDFVersion	存储版本号的整形指针（不能为空）	输出

➤ 返回值

返回值 { 0 成功。
非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys.h、mi_md.h、mi_od.h、mi_vg.h、mi_vdf.h
- 库文件：libOD_LINUX.a、libMD_LINUX.a、libVG_LINUX.a、libmi_vdf.a

※ 注意

无。

➤ 举例

无。

➤ 相关主题

无。

2.2.13 MI_VDF_GetDebugInfo

➤ 功能

获取 MD/OD/VG 库的调试信息。

➤ 语法

```
MI_S32 MI_VDF_GetDebugInfo(MI_VDF_CHANNEL VdfChn, MI_VDF_DebugInfo_t *pstDebugInfo);
```

➤ 形参

参数名称	描述	输入/输出
VdfChn	已经创建的视频侦测通道号。	输入
pstDebugInfo	存储 VDF 调试信息的整形指针（不能为空）	输出

➤ 返回值

返回值 { 0 成功。
非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys.h、mi_md.h、mi_od.h、mi_vg.h、mi_vdf.h
- 库文件：libOD_LINUX.a、libMD_LINUX.a、libVG_LINUX.a、libmi_vdf.a

※ 注意

- 目前只支持返回 VG 的调试信息，当 VdfChn 的工作模式是 VG 时，才有效。

➤ 举例

无。

- 相关主题
无。

3. 数据类型

3.1. 结构体列表

相关数据类型、数据结构、联合体定义如下：

结构体	定义
MI_VDF_WorkMode_e	定义 VDF 工作模式的枚举类型
MI_VDF_Color_e	定义视频侦测通道输入源的枚举类型
MI_VDF_ODWindow_e	定义 OD 时画面的子窗口数量的枚举类型
MI_MD_Result_t	定义 MD 结果的结构体
MI_OD_Result_t	定义 OD 结果的结构体
MI_VG_Result_t	定义 VG 结果的结构体
MI_VDF_Result_t	定义 VDF 工作模式对应结果的结构体
MI_VDF_MdAttr_t	定义 MD 通道属性的结构体
MI_VDF_OdAttr_t	定义 OD 通道属性的结构体
MI_VDF_VgAttr_t	定义 VG 通道属性的结构体
MI_VDF_ChnAttr_t	定义通道工作模式属性的结构体
MDRST_STATUS_t	定义侦测子窗口区域运动检测结果的结构体
MI_MD_ResultSize_t	定义 MD 结果长度的结构体

3.2. VDF 结构体说明

3.2.1 MI_VDF_WorkMode_e

➤ 说明

定义 VDF 工作模式的枚举类型。

➤ 定义

```
typedef enum
{
    E_MI_VDF_WORK_MODE_MD = 0,
    E_MI_VDF_WORK_MODE_OD,
    E_MI_VDF_WORK_MODE_VG,
    E_MI_VDF_WORK_MODE_MAX
}MI_VDF_WorkMode_e;
```

➤ 成员

成员名称	描述
E_MI_VDF_WORK_MODE_MD	MD 工作模式
E_MI_VDF_WORK_MODE_OD	OD 工作模式
E_MI_VDF_WORK_MODE_VG	VG 工作模式
E_MI_VDF_WORK_MODE_MAX	工作模式的错误码标识

※ 注意事项
无。

➤ 相关数据接口及类型
无。

3.2.2 MI_VDF_Color_e

➤ 说明

定义视频侦测通道输入源的枚举类型

➤ 定义

```
typedef enum
{
    E_MI_VDF_COLOR_Y = 1,
    E_MI_VDF_COLOR_MAX
} MI_VDF_Color_e;
```

➤ 成员

成员名称	描述
E_MI_VDF_COLOR_Y	视频侦测通道输入源类型的正确标识
E_MI_VDF_COLOR_MAX	视频侦测通道输入源类型的错误码标识

※ 注意事项
无。

➤ 相关数据接口及类型
无。

3.2.3 MI_VDF_ODWindow_e

➤ 说明

定义OD时画面的子窗口数量的枚举类型

➤ 定义

```
typedef enum
{
    E_MI_VDF_ODWINDOW_1X1 = 0,
    E_MI_VDF_ODWINDOW_2X2,
    E_MI_VDF_ODWINDOW_3X3,
```

```
E_MI_VDF_ODWINDOW_MAX  
} MI_VDF_ODWindow_e;
```

➤ 成员

成员名称	描述
E_MI_VDF_ODWINDOW_1X1	OD 画面分割为 1 个子窗口
E_MI_VDF_ODWINDOW_2X2	OD 画面分割为 2x2 个子窗口
E_MI_VDF_ODWINDOW_3X3	OD 画面分割为 3x3 个子窗口
E_MI_VDF_ODWINDOW_MAX	OD 画面子窗口数量的错误码标识

※ 注意事项
无。

➤ 相关数据接口及类型
无。

3.2.4 MI_MD_Result_t

➤ 说明

定义 MD 结果的结构体。

➤ 定义

```
typedef struct MI_MD_Result_s  
{  
    MI_U64 u64Pts;           //The PTS of Image  
    MI_U8 u8Enable;          //=1 表明该结果值有效  
    MI_MD_ResultSize_t stSubResultSize;  
    MDRST_STATUS_t* pstMdResultStatus; //The MD result of Status  
    MDSAD_DATA_t* pstMdResultSad; //The MD result of SAD  
    MDOBJ_DATA_t* pstMdResultObj; //The MD result of Obj  
}MI_MD_Result_t;
```

➤ 成员

成员名称	描述
u64Pts	图像显示时间
u8Enable	表明该通道是否使能
u8Reading	表明该结果正在被应用层读取
stSubResultSize	描述 MD 返回的 Sad、Obj、ReasultStauts 子结构的 Size
pstMdResultStatus	描述 MD 各区域是否检测到运动
pstMdResultSad	描述 MD 的 Sad 值
pstMdResultObj	描述 MD 的 CCL 值

※ 注意事项
无。

➤ 相关数据类型及接口
无。

3.2.5 MI_OD_Result_t

➤ 说明

定义 OD 结果的结构体。

➤ 定义

```
typedef struct MI_OD_Result_s
{
    MI_U8 u8Enable;
    MI_U8 u8WideDiv;    //The number of divisions of window in horizontal direction
    MI_U8 u8HightDiv;   //The number of divisions of window in vertical direction
    MI_U8 u8DataLen;    //OD detect result readable size
    MI_U64 u64Pts;      //The PTS of Image
    MI_S8 u8RgnAlarm[3][3]; //The OD result of the sub-window
    MI_S8 s8OdStatus;    //The OD detect status
}MI_OD_Result_t;
```

➤ 成员

成员名称	描述
u8Enable	表明该通道是否使能
u8WideDiv	获取 OD 在水平方向的窗口数量
u8HightDiv	获取 OD 在垂直方向的窗口数量
u8DataLen	设置 OD 测试结果的可读大小
u64Pts	图像显示时间
u8RgnAlarm[3][3]	OD 子窗口信息的结果 0: 视窗没有被遮挡 1: 视窗被遮挡 2: 视窗特征不足 -1: 失败
s8OdStatus	1. s8OdStatus = 0 则 No od detected 2. s8OdStatus = 1 则 Od detected 3. s8OdStatus = -1 则 Run od error

※ 注意事项

若 user 需要获取 s8OdStatus 这个值，只需关注状态值 0 或者 1 来进行条件判断。当其为-1 时并不需要关注，OD 已经 run 失败了所以自然就 [MI_VDF_GetResult](#) 会失败，从而不再拿到 s8OdStatus 值。

➤ 相关数据类型及接口

无。

3.2.6 MI_VG_Result_t

➤ 说明

定义 VG 结果的结构体。

➤ 定义

```
typedef struct _MI_VgResult_t
```

```
{
    int32_t alarm[MAX_NUMBER];
    int32_t alarm_cnt;
    MI\_VgBoundingBox\_t bounding_box[20];
} MI_VgResult_t;
```

➤ 成员

成员名称	描述
alarm	描述 VG 的检测结果 #define MAX_NUMBER 4
alarm_cnt	描述 VG 的警报次数
bounding_box	描述触发 VG 警报的物体信息

※ 注意事项
无。

➤ 相关数据类型及接口
无。

3.2.7 MI_VDF_Result_t

➤ 说明

定义 VDF 工作模式对应结果的结构体。

➤ 定义

```
typedef struct MI_VDF_Result_s
{
    MI\_VDF\_WorkMode\_e enWorkMode;
    VDF_RESULT_HANDLE handle;
    union
    {
        MI\_MD\_Result\_t stMdResult;
        MI\_OD\_Result\_t stOdResult;
        MI\_VG\_Result\_t stVgResult;
    };
}MI_VDF_Result_t;
```

➤ 成员

成员名称	描述
enWorkMode	VDF 工作模式（MD/OD/VG）
handle	保存结果的 handle
stMdResult	描述 MD 结果的结构体
stOdResult	描述 OD 结果的结构体
stVgResult	描述 VG 结果的结构体

※ 注意事项
无。

➤ 相关数据类型及接口

无。

3.2.8 MI_VDF_MdAttr_t

➤ 说明

定义 MD 通道属性的结构体

➤ 定义

```
typedef struct MI_VDF_MdAttr_s
{
    MI_U8 u8Enable;
    MI_U8 u8MdBufCnt;
    MI_U8 u8VDFIntvl;
    MI_U32 u32RstBufSize;
    MI_MD_ResultSize_t stSubResultSize;
    MDCCL_ctrl_t ccl_ctrl;
    MI_MD_static_param_t stMdStaticParamsIn;
    MI_MD_param_t stMdDynamicParamsIn;
}MI_VDF_MdAttr_t;
```

➤ 成员

成员名称	描述
u8Enable	表明该通道是否使能
u8MdBufCnt	设置可以缓存的 MD 结果数量 MD 结果缓存个数取值范围: [1, 8] 静态属性
u8VDFIntvl	侦测间隔取值范围: [0, 29], 以帧为单位 动态属性
u32RstBufSize	MD 返回结果的总大小
stSubResultSize	MD 返回结果各子结构体 (struct MI_MD_ResultSize_s) (Sad 值、CCL 值、ResultStatus) 的大小
ccl_ctrl	ccl_ctrl 属性设置
stMdStaticParamsIn	MD 静态属性参数设置
stMdDynamicParamsIn	MD 动态属性参数设置

※ 注意事项

无。

➤ 相关数据类型及接口

无。

3.2.9 MI_VDF_OdAttr_t

➤ 说明

定义 OD 通道属性的结构体

➤ 定义

```
typedef struct MI_VDF_OdAttr_s
{
    MI_U8 u8Enable;
```



```
MI_U8 u8OdBufCnt;
MI_U8 u8VDFIntvl;
MI_U32 u32RstBufSize;
MI_OD_static_param_t stOdStaticParamsIn;
MI_OD_param_t stOdDynamicParamsIn;
}MI_VDF_OdAttr_t;
```

➤ 成员

成员名称	描述
u8Enable	表明该通道是否使能
u8OdBufCnt	OD 结果缓存个数取值范围: [1, 16] 静态属性
u8VDFIntvl	侦测间隔取值范围: [0, 29], 以帧为单位 动态属性
u32RstBufSize	OD 返回结果的总大小
stOdDynamicParamsIn	OD 动态属性参数设置
stOdStaticParamsIn	OD 静态属性参数设置

※ 注意事项
无。

➤ 相关数据类型及接口
无。

3.2.10 MI_VDF_VgAttr_t

➤ 说明

定义 VG 通道属性的结构体

➤ 定义

```
typedef struct MI_VDF_VgAttr_s
{
    MI_U8 u8Enable;
    MI_U8 u8VgBufCnt;
    MI_U8 u8VDFIntvl;
    MI_U32 u32RstBufSize;

    MI_U16 width;
    MI_U16 height;
    MI_U16 stride;

    float object_size_thd;
    uint8_t indoor;
    uint8_t function_state;
    uint16_t line_number;
    MI_VgLine_t line[4];
    MI_VgRegion_t vg_region;

    MI_VgSet_t stVgParamsIn;
} MI_VDF_VgAttr_t;
```

➤ 成员

成员名称	描述
u8Enable	表明该通道是否使能
u8VgBufCnt	VG 结果缓存个数取值范围: [1, 8] 静态属性
u8VDFIntvl	侦测间隔取值范围: [0, 29], 以帧为单位 动态属性
u32RstBufSize	VG 返回结果的总大小
width	图像的宽度
height	图像的高度
stride	图像的 stride
object_size_thd	决定滤除物体占感兴趣区域的百分比大小 (若 object_size_thd = 1, 表示有物体面积小于图像画面中感兴趣区域的百分之一则会忽略不计算。)
indoor	室内或者室外, 1-室内, 0-室外
function_state	设定虚拟线段与区域入侵侦测模式类型: 1) VG_VIRTUAL_GATE, 表示模式为虚拟线段; 2) VG_REGION_INVASION, 表示模式为区域入侵
line_number	设定虚拟线段的数目, 范围: [1-4]
line[4]	表明虚拟线段的结构体, 最多可设置 4 条
vg_region	表明区域入侵的相关参数
stVgParamsIn	Vg 属性参数结构体, 无需设置, 由 API 返回

※ 注意事项
无。

➤ 相关数据类型及接口
无。

3.2.11 MI_VDF_ChnAttr_t

➤ 说明
定义通道工作模式属性的结构体。

➤ 定义

```
typedef struct MI_VDF_ChnAttr_s
{
    MI_VDF_WorkMode_e enWorkMode;
    union
    {
        MI_VDF_MdAttr_t stMdAttr;
        MI_VDF_OdAttr_t stOdAttr;
        MI_VDF_VgAttr_t stVgAttr;
    };
}MI_VDF_ChnAttr_t;
```

➤ 成员

成员名称	描述
enWorkMode	工作模式(移动侦测,遮挡侦测,电子围栏) 静态属性

成员名称	描述
stMdAttr	移动侦测属性
stOdAttr	遮挡侦测属性
stVgAttr	电子围栏属性

※ 注意事项
无。

➤ 相关数据类型及接口
无。

3.2.12 MDRST_STATUS_t

➤ 说明
定义侦测子窗口区域运动检测结果的结构体

➤ 定义

```
typedef struct MDRST_STATUS_s
{
    MI_U8 *paddr;
} MDRST_STATUS_t;
```

➤ 成员

成员名称	描述
paddr	指向运动检测状态的 buf，每个区域占用 1 字节 0-区块未检测运动，255-区块检测到运动

※ 注意事项
无。

➤ 相关数据类型及接口
无。

3.2.13 MI_MD_ResultSize_t

➤ 说明
定义 MD 结果长度的结构体。

➤ 定义

```
typedef struct MI_MD_ResultSize_s
{
    MI_U32 u32RstStatusLen;
    MI_U32 u32RstSadLen;
    MI_U32 u32RstObjLen;
}MI_MD_ResultSize_t;
```

➤ 成员

成员名称	描述
u32RstStatusLen	描述 MD 检测到运动状态值的长度

u32RstSadLen	描述 MD 的 Sad 值的长度
u32RstObjLen	描述 MD 的 CCL 值的长度

※ 注意事项
无。

➤ 相关数据类型及接口
无。

3.3. MD 结构体列表

数据类型	定义
MDMB_MODE_e	宏块的大小枚举值
MDSAD_OUT_CTRL_e	SAD 输出格式的枚举值
MDALG_MODE_e	CCL 连通区域的运算模式枚举值，可依前景结果或者 SAD 结果做 CCL 运算
MDCCL_ctrl_t	控制 CCL 运行的参数结构
MDPreproc_ctrl_t	控制 MI_MD_Preproc 运行的参数结构
MDblock_info_t	ROI 坐标结构
MDPoint_t	坐标结构
MDROI_t	MD 侦测区域结构
MDSAD_DATA_t	MI_MD_ComputeImageSAD() 函数输出的结构
MDOBJ_t	定义连通区域的信息：面积及最小包围矩形的坐标位置
MDOBJ_DATA_t	CCL 输出的结构
MI_MD_IMG_t	移动侦测的图像来源结构，分为实体与虚拟的内存地址指针。
MI_MD_IMG64_t	移动侦测的图像来源结构，分为 64bit 实体内存地址与虚拟的内存地址指针。
MI_MD_static_param_t	MD 静态参数设置结构
MI_MD_param_t	MD 动态参数设置结构

3.4. MD 结构体说明

3.4.1 MDMB_MODE_e

➤ 说明

宏块的大小枚举值。

➤ 定义

```
typedef enum MDMB_MODE_E
{
    MDMB_MODE_MB_4x4    = 0x0,
    MDMB_MODE_MB_8x8    = 0x1,
    MDMB_MODE_MB_16x16  = 0x2,
```

```
MDMB_MODE_BUTT
} MDMB_MODE_e;
```

➤ 成员

成员名称	描述
MDMB_MODE_MB_4x4	使用 4x4 宏块
MDMB_MODE_MB_8x8	使用 8x8 宏块
MDMB_MODE_MB_16x16	使用 16x16 宏块

3.4.2 MDSAD_OUT_CTRL_e

➤ 说明

SAD 输出格式的枚举值。

➤ 定义

```
typedef enum MDSAD_OUT_CTRL_E
{
    MDSAD_OUT_CTRL_16BIT_SAD = 0x0,
    MDSAD_OUT_CTRL_8BIT_SAD  = 0x1,
    MDSAD_OUT_CTRL_BUTT
} MDSAD_OUT_CTRL_e;
```

➤ 成员

成员名称	描述
MDSAD_OUT_CTRL_16BIT_SAD	16 bit 输出
MDSAD_OUT_CTRL_8BIT_SAD	8 bit 输出

3.4.3 MDALG_MODE_e

➤ 说明

CCL 连通区域的运算模式枚举值，可依前景结果或者 SAD 结果做 CCL 运算。

➤ 定义

```
typedef enum MDALG_MODE_E
{
    MDALG_MODE_FG      = 0x0,
    MDALG_MODE_SAD     = 0x1,
    MDALG_MODE_FRAMEDIFF = 0x2,
    MDALG_MODE_BUTT
} MDALG_MODE_e;
```

➤ 成员

成员名称	描述
MDALG_MODE_FG	前景模式
MDALG_MODE_SAD	SAD 模式
MDALG_MODE_FRAMEDIFF	FrameDifference 模式

3.4.4 MDCCL_ctrl_t

➤ 说明

控制 CCL 运行的参数结构。

➤ 定义

```
typedef struct MDCCL_ctrl_s
{
    uint16_t u16InitAreaThr;
    uint16_t u16Step;
} MDCCL_ctrl_t;
```

➤ 成员

成员名称	描述
u16InitAreaThr	区域面积的门坎值(这个值不是一定会上, 推荐值设定 8)
u16Step	每提高一次门坎值的提升值(这个值不是一定会上, 推荐值设定 2)

3.4.5 MDPreproc_ctrl_t

➤ 说明

控制 MI_MD_Preproc 运行的参数结构。

➤ 定义

```
typedef struct MDPreproc_ctrl_s
{
    uint16_t u16Md_rgn_size;
    uint16_t u16Align;
} MDPreproc_ctrl_t;
```

➤ 参数

参数	说明
u16Md_rgn_size	移动区域的大小范围门坎值, 小于此门坎则不计算该区域为动态区域
u16Align	对应 CNN model 的 align 限制(HC/HD:32ss align, FD:64align)

3.4.6 MDblock_info_t

➤ 描述

描述一个 block 的 ROI 坐标结构。

➤ 定义

```
typedef struct MDblock_info_s
{
    uint16_t st_x;
    uint16_t st_y;
    uint16_t end_x;
    uint16_t end_y;
} MDblock_info_t;
```

➤ 参数

参数	说明
st_x	ROI 的左上 x 坐标
st_y	ROI 的左上 y 坐标
end_x	ROI 的右下 x 坐标
end_y	ROI 的右下 y 坐标

3.4.7 MDPoint_t

➤ 说明

坐标结构。

➤ 定义

```
typedef struct MDPoint_s
{
    uint16_t x;
    uint16_t y;
} MDPoint_t;
```

➤ 成员

成员名称	描述
x	X 坐标
y	Y 坐标

3.4.8 MDROI_t

➤ 说明

MD 侦测区域结构。

➤ 定义

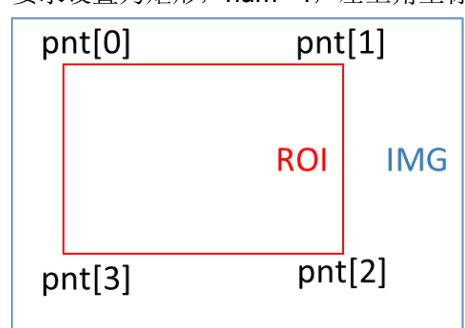
```
typedef struct MDROI_s
{
    uint8_t num;
    MDPoint\_t pnt[8];
} MDROI_t;
```

➤ 成员

成员名称	描述
num	MD 侦测区域点数，目前仅支持设为 4 点
pnt[8]	四点坐标(须设为矩形)

※ 注意事项

要求设置为矩形，num=4，左上角坐标顺时针依序设置四个点，如图示。



3.4.9 MDSAD_DATA_t

➤ 说明

MI_MD_ComputeImageSAD 函式输出的结构。

➤ 定义

```
typedef struct MDSAD_DATA_s
{
    void *paddr;
    uint32_t stride;
    MDSAD\_OUT\_CTRL\_e enOutCtrl;
} MDSAD_DATA_t;
```

➤ 成员

成员名称	描述
paddr	存放 SAD 结果的内存地址指针
stride	Image stride
enOutCtrl	SAD 输出格式的枚举值

3.4.10 MDOBJ_t

➤ 说明

定义连通区域的信息：面积及最小包围矩形的坐标位置。

➤ 定义

```
typedef struct MDOBJ_s
{
    uint32_t u32Area;
    uint16_t u16Left;
    uint16_t u16Right;
    uint16_t u16Top;
    uint16_t u16Bottom;
} MDOBJ_t;
```

➤ 成员

成员名称	描述
u32Area	单一连通区域的像素总数
u16Left	最小矩形的左上 x 坐标
u16Right	最小矩形的右下 x 坐标
u16Top	最小矩形的左上 y 坐标
u16Bottom	最小矩形的右下 y 坐标

3.4.11 MDOBJ_DATA_t

➤ 说明

MI_MD_CCL 输出的结构。

➤ 定义

```
typedef struct MDOBJ_DATA_s
{
    uint8_t u8RegionNum;
    MDOBJ_t *astRegion;
    uint8_t indexofmaxobj;
    uint32_t areaofmaxobj;
    uint32_t areaoftotalobj;
} MDOBJ_DATA_t;
```

➤ 成员

成员名称	描述
u8RegionNum	连通区域数量

成员名称	描述
astRegion	连通区域的信息：面积及最小包围矩形的坐标位置最大容许数量为 255
indexofmaxobj	最大面积的连通区域索引值
areaofmaxobj	最大面积的连通区域面积值
areaoftotalobj	所有连通区域的面积和

3.4.12 MI_MD_IMG_t

➤ 说明

移动侦测的图像来源结构，分为物理与虚拟的内存地址指针。

➤ 定义

```
typedef struct MI_MD_IMG_s
{
    void *pu32PhyAddr;
    uint8_t *pu8VirAddr;
} MI_MD_IMG_t;
```

➤ 成员

成员名称	描述
pu32PhyAddr	物理内存地址指针，若无请填写 NULL
pu8VirAddr	虚拟内存地址指针

3.4.13 MI_MD_IMG64_t

➤ 说明

移动侦测的图像来源结构，分为 64bit 物理内存地址与虚拟的内存地址指针。

➤ 定义

```
typedef struct MI_MD_IMG64_s
{
    uint64_t u64PhyAddr;
    uint8_t *pu8VirAddr;
} MI_MD_IMG64_t;
```

➤ 成员

成员名称	描述
u64PhyAddr	物理内存地址
pu8VirAddr	虚拟内存地址指针

3.4.14 MI_MD_static_param_t

➤ 说明

MD 静态参数设置结构。

➤ 定义

```
typedef struct MI_MD_static_param_s
{
    uint16_t width;
    uint16_t height;
    uint8_t color;
    uint32_t stride;
    MDMB_MODE_e mb_size;
    MDSAD_OUT_CTRL_e sad_out_ctrl;
    MDROI_t roi_md;
    MDALG_MODE_e md_alg_mode;
} MI_MD_static_param_t;
```

➤ 成员

成员名称	描述
width	输入图像宽
height	输入图像高
stride	输入图像的 stride
color	MD 输入图像的类型
mb_size	宏块的大小枚举值
sad_out_ctrl	SAD 输出格式的枚举值
roi_md	MD 侦测区域结构
md_alg_mode	CCL 连通区域的运算模式枚举值

3.4.15 MI_MD_param_t

➤ 说明

MD 动态参数设置结构。

➤ 定义

```
typedef struct MI_MD_param_s
{
    uint8_t sensitivity;
    uint16_t learn_rate;
    uint32_t md_thr;
    uint32_t obj_num_max;
    uint8_t LSD_open;
```

} MI_MD_param_t;

➤ 成员

成员名称	描述
sensitivity	算法灵敏度, 范围[10,20,30,.....100], 值越大越灵敏, 输入的灵敏度如非 10 的倍数, 当运算后反馈, 有可能不为当初输入的数值, 会有 +-1 之偏差。仅于 FG 模式下有效。
learn_rate	随不同模式而有不同设定标准 ✓ MDALG_MODE_FG : 单位毫秒, 范围[1000,30000], 用于控制前端物体停止运动多久时, 才作为背景画面 ✓ MDALG_MODE_SAD : 范围[1, 255], 背景更新的权重比, 建议设定值:128
md_thr	当呼叫 MI_MD_CCL 连通组件标记的运算时, 作为判定移动有效的门坎值, 随不同模式而有不同设定标准 ✓ MDALG_MODE_FG : 设置为百分比(%), 范围[0, 99], 判断 MB 是否警报的门坎值, ex. 若 MB size 8*8, md_thr=50, 当 MB 中有超过 32 个以上的 pixel 为前景则此 MB 判为有效。 ✓ MDALG_MODE_SAD : 设置为 pixel 差值, 范围[0, 255], 判断 MB 是否警报的门坎值, ex. 若 MB size 8*8, md_thr=50, 当 MB 中平均的 pixel 差值超过 50, 则此 MB 判为有效。
obj_num_max	CCL 的连通区域数量限制值
LSD_open	决定是否作用 MI_MD_LightSwitchDetect。范围[0,1], 若在 LSD_open=0 的情况下调用 LightSwitchDetect() 函数, 则无作用。

3.5. OD 结构体列表

枚举	
MI_OD_WIN_STATE	OD 检测窗口的结果
ODColor_e	OD 数据源输入的类型
ODWindow_e	OD 检测窗口的类型
结构	
ODPoint_t	坐标结构
ODROI_t	OD 侦测区域结构
MI_OD_IMG_t	遮挡侦测的图像来源结构, 分为实体与虚拟的内存地址指针。
MI_OD_IMG64_t	遮挡侦测的图像来源结构, 分为 64bit 实体内存地址与虚拟的内存地址指针。
MI_OD_static_param_t	OD 静态参数设置结构
MI_OD_param_t	OD 动态参数设置结构

3.6. OD 结构体说明

3.6.1 MI_OD_WIN_STATE

➤ 说明

OD 检测窗口的结果。

➤ 定义

```
typedef enum _MI_OD_WIN_STATE
{
    MI_OD_WIN_STATE_NON_TAMPER = 0,
    MI_OD_WIN_STATE_TAMPER = 1,
    MI_OD_WIN_STATE_NO_FEATURE = 2,
    MI_OD_WIN_STATE_FAIL = -1,
} MI_OD_WIN_STATE;
```

➤ 成员

成员名称	描述
MI_OD_WIN_STATE_NON_TAMPER	窗口没遮挡
MI_OD_WIN_STATE_TAMPER	窗口被遮挡
MI_OD_WIN_STATE_NO_FEATURE	窗口特征不足
MI_OD_WIN_STATE_FAIL	失败

3.6.2 ODColor_e

➤ 说明

OD 数据源输入的类型。

➤ 定义

```
typedef enum
{
    OD_Y = 1,
    OD_COLOR_MAX
} ODColor_e;
```

➤ 成员

成员名称	描述
OD_Y	YUV 数据源中的 y 分量
OD_COLOR_MAX	输入图像类型的最大值

3.6.3 ODWindow_e

➤ 说明

OD 检测窗口的类型，推荐值为 OD_WINDOW_3X3，用于测试。

➤ 定义

```
typedef enum
{
    OD_WINDOW_1X1 = 0,
    OD_WINDOW_2X2,
    OD_WINDOW_3X3,
    OD_WINDOW_MAX
} ODWindow_e;
```

➤ 成员

成员名称	描述
OD_WINDOW_1X1	1 个窗口
OD_WINDOW_2X2	4 个窗口
OD_WINDOW_3X3	9 个窗口
OD_WINDOW_MAX	窗口类型的最大值

3.6.4 ODPoint_t

➤ 说明

坐标结构。

➤ 定义

```
typedef struct ODPoint_s
{
    uint16_t x;
    uint16_t y;
} ODPoint_t;
```

➤ 成员

成员名称	描述
x	X 坐标
y	Y 坐标

3.6.5 ODROI_t

➤ 说明

OD 侦测区域结构。

➤ 定义

```
typedef struct ODROI_s
{

```

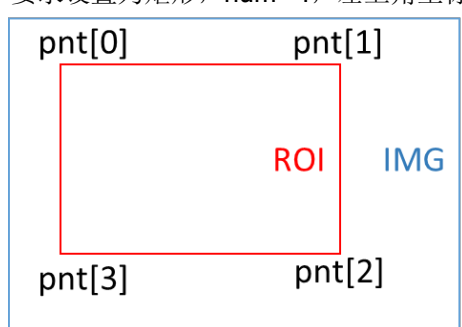
```
uint8_t num;
ODPoint_t pnt[8];
} ODRoi_t;
```

➤ 成员

成员名称	描述
num	OD 侦测区域点数，目前仅支持设为 4 点
pnt[8]	四点坐标(须设为矩形)

➤ 注意

要求设置为矩形，num=4，左上角坐标顺时针依序设置四个点，如图示。



3.6.6 MI_OD_IMG_t

➤ 说明

遮挡侦测的图像来源结构，分为实体与虚拟的内存地址指针。

➤ 定义

```
typedef struct MI_OD_IMG_s
{
    void *pu32PhyAddr;
    uint8_t *pu8VirAddr;
} MI_OD_IMG_t;
```

➤ 成员

成员名称	描述
pu32PhyAddr	实体的内存地址指针
pu8VirAddr	虚拟的内存地址指针

3.6.7 MI_OD_IMG64_t

➤ 说明

遮挡侦测的图像来源结构，分为 64bit 实体内存地址与虚拟的内存地址指针。

➤ 定义

```
typedef struct MI_OD_IMG64_s
{
    uint64_t u64PhyAddr;
    uint8_t *pu8VirAddr;
} MI_OD_IMG64_t;
```

➤ 成员

成员名称	描述
u64PhyAddr	实体的内存地址
pu8VirAddr	虚拟的内存地址指针

3.6.8 MI_OD_static_param_t

➤ 说明

OD 静态参数设置结构。

➤ 定义

```
typedef struct MI_OD_static_param_s
{
    uint16_t inImgW;
    uint16_t inImgH;
    uint32_t inImgStride;
    ODColor\_e nClrType;
    ODWindow\_e div;
    ODROI\_t roi_od;
    int32_t alpha;
    int32_t M;
    int32_t MotionSensitivity;
} MI_OD_static_param_t;
```

➤ 成员

成员名称	描述
inImgW	输入图像宽
inImgH	输入图像高
inImgStride	输入图像 stride
nClrType	OD 输入图像的类型
div	OD 检测窗口的类型
roi_od	OD 侦测区域结构
alpha	控制产生参考图像的学习速率
M	多少张图像更新一次参考图像
MotionSensitivity	移动敏感度设置

※ 注意事项

- 设置范围 **alpha** : 0~10, 建议设置 2, 不建议更动。
- 设置范围 **MotionSensitivity**: 0~5, 设 5 表示对轻微的晃动都很敏感, 容易发报; 设 0 表示对轻微晃动的宽容性比较好, 不会发报, 这边指的轻微晃动是风吹摇曳之类的, 建议初始可设 5。
- **M** 建议设置 120, 不建议更动。

3.6.9 MI_OD_param_t

➤ 说明

OD 动态参数设置结构。

➤ 定义

```
typedef struct MI_OD_param_s
{
    int32_t thd_tamper;
    int32_t tamper_blk_thd;
    int32_t min_duration;
} MI_OD_param_t;
```

➤ 成员

成员名称	描述
thd_tamper	图像差异比例门坎值
tamper_blk_thd	图像被遮挡区域数量门坎值
min_duration	图像差异持续时间门坎值, 单位是 frame cnt

※ 注意事项

- 设置范围 **thd_tamper** : 0~10。若 **thd_tamper**=3, 表示超过 70%画面遮挡即发报。
- 设置范围 **tamper_blk_thd** : 对应 **MI_OD_Init** 的窗口类型参数, 若为 **OD_WINDOW_3X3**, 则 **tamper_blk_thd** 最多不可超过 9, 即 1~9。
- 例如 **MI_OD_Init** 的窗口类型参数为 **OD_WINDOW_3X3** (9 个子区域) **tamper_blk_thd** 值为 4 时, 当被遮挡的子区域的数量达到 4 个才触发 **MI_OD_Run** 的返回值为 1。
- **min_duration** 数值越大, 检测到被遮挡所需的时间越长。
- **MI_OD_Run** 的灵敏度可以通过设置 **tamper_blk_thd** 和 **min_duration** 来调节。对应高中低的推荐值如下:

参数名	高	中	低
tamper_blk_thd	2	4	8
min_duration	5	15	30

3.7. VG 结构体列表

枚举	
VgFunction	侦测模式的枚举值
VgRegion_Dir	区域入侵的方向的枚举值

VgSize_Sensitively	灵敏度参数结构
VgDirection_State	区域入侵的侦测规则枚举值
结构	
MI_VG_Point_t	坐标点对应的结构
MI_VgLine_t	描述虚拟线段和方向的结构
MI_VgRegion_t	描述设置区域入侵的结构
MI_VgSet_t	Vg 对应参数设置的结构
MI_VgResult_t	Vg 检测结果对应的结构
MI_VgBoundingBox_t	Vg 检测结果对应的物件大小结构
MI_VgDetectThd	Vg 建立背景信息的参数设置的结构

3.8. VG 结构体说明

3.8.1 VgFunction

➤ 说明

侦测模式的枚举值。

➤ 定义

```
typedef enum _VgFunction
{
    VG_VIRTUAL_GATE    = 2,
    VG_REGION_INVASION = 3
} VgFunction;
```

➤ 成员

成员名称	描述
VG_VIRTUAL_GATE	表示模式为虚拟线段
VG_REGION_INVASION	表示模式为区域入侵

3.8.2 VgRegion_Dir

➤ 说明

区域入侵的方向的枚举值。

➤ 定义

```
typedef enum _VgRegion_Dir
{
    VG_REGION_ENTER    = 0,
    VG_REGION_LEAVING  = 1,
    VG_REGION_CROSS    = 2
} VgRegion_Dir;
```

➤ 成员

成员名称	描述
VG_REGION_ENTER	表示要进入警报区域才触发警报
VG_REGION_LEAVING	表示要离开警报区域才触发警报
VG_REGION_CROSS	表示只要穿越警报区域就触发警报

3.8.3 VgSize_Sensitively

➤ 说明

灵敏度参数结构。

➤ 定义

```
typedef enum _VgSize_Sensitively
{
    VG_SENSITIVELY_MIN    = 0,
    VG_SENSITIVELY_LOW    = 1,
    VG_SENSITIVELY_MIDDLE = 2,
    VG_SENSITIVELY_HIGH   = 3,
    VG_SENSITIVELY_MAX    = 4
} VgSize_Sensitively;
```

➤ 成员

成员名称	描述
VG_SENSITIVELY_MIN	表示只侦测大于 30%影像大小的物体
VG_SENSITIVELY_LOW	表示只侦测大于 10%影像大小的物体
VG_SENSITIVELY_MIDDLE	表示只侦测大于 5%影像大小的物体
VG_SENSITIVELY_HIGH	表示只侦测大于 1%影像大小的物体
VG_SENSITIVELY_MAX	表示只侦测大于 0.5%影像大小的物体

3.8.4 VgDirection_State

➤ 说明

区域入侵的侦测规则枚举值。

➤ 定义

```
typedef enum _VgDirection_State
{
    VG_SPEC_DIRECTION_CLOSE = 0,
    VG_SPEC_DIRECTION_OPEN  = 1
} VgDirection_State;
```

➤ 成员

成员名称	描述
VG_SPEC_DIRECTION_CLOSE	关闭结果会区分进入和离开区的功能 (设定 VG_REGION_CROSS 前提下, 只侦测警报, 无判断方向功能。)
VG_SPEC_DIRECTION_OPEN	开启结果会区分进入和离开区的功能 (设定 VG_REGION_CROSS 前提下, 可判断出警报时的方向是进入还是离开。)

3.8.5 MI_VG_Point_t

➤ 说明

坐标点对应的结构。

➤ 定义

```
typedef struct _VG_Point_t
{
    int32_t x;
    int32_t y;
} MI_VG_Point_t;
```

➤ 成员

成员名称	描述
x	X 坐标
y	Y 坐标

3.8.6 MI_VgLine_t

➤ 说明

描述虚拟线段和方向的结构。

➤ 定义

```
typedef struct _VG_Line_t
{
    MI_VG_Point_t px; //point x
    MI_VG_Point_t py; //point y
    MI_VG_Point_t pdx; //point direction x
    MI_VG_Point_t pdy; //point direction y
} MI_VgLine_t;
```

➤ 成员

成员名称	描述
------	----

px	第一个线段点
py	第二个线段点
pdx	第一个方向点
pdY	第二个方向点

3.8.7 MI_VgRegion_t

➤ 说明

描述设置区域入侵的结构。

➤ 定义

```
typedef struct _VG_Region_t
{
    MI_VG_Point_t p_one;    //point one
    MI_VG_Point_t p_two;    //point two
    MI_VG_Point_t p_three;  //point three
    MI_VG_Point_t p_four;   //point four

    int32_t region_dir;     //Region direction;
    int32_t spec_dir_state;
} MI_VgRegion_t;
```

➤ 成员

成员名称	描述
p_one	描述区域的第一个点
p_two	描述区域的第二个点
p_three	描述区域的第三个点
p_four	描述区域的第四个点
region_dir	设定区域入侵的方向
spec_dir_state	设定区域入侵在 region_dir=VG_REGION_CROSS 时，是否区分进入和离开

3.8.8 MI_VgSet_t

➤ 说明

Vg 对应参数设置的结构。

➤ 定义

```
typedef struct _MI_VgSet_t
{
    //Common Information
    float object_size_thd;
```

```

uint16_t line_number;
uint8_t indoor;

//Line info
MI_VG_Point_t fp[4]; //First point
MI_VG_Point_t sp[4]; //Second point
MI_VG_Point_t fdp[4]; //First direction point
MI_VG_Point_t sdp[4]; //Second direction point

//Function
uint8_t function_state;

//Region info
MI_VG_Point_t first_p; //First point
MI_VG_Point_t second_p; //Second point
MI_VG_Point_t third_p; //Third point
MI_VG_Point_t fourth_p; //Fourth point

//Region direction
uint8_t region_direction;

//Magic_number
int32_t magic_number;

int32_t region_spdir_state;
} MI_VgSet_t;

```

➤ 成员

成员名称	描述
object_size_thd	决定滤除物体占感兴趣区域的百分比阈值
line_number	设定虚拟线段的数目，范围：[1-4]
indoor	室内或者室外，1-室内，0-室外
fp[4]	第一个线段点数组
sp[4]	第二个线段点数组
fdp[4]	第三个线段点数组
sdp[4]	第四个线段点数组
function_state	设定虚拟线段、区域入侵侦测模式
first_p	入侵区域第一个点
second_p	入侵区域第二个点
third_p	入侵区域第三个点
fourth_p	入侵区域低四个点

region_direction	表明区域入侵的相关参数
magic_number	Magic Number
region_spdir_state	入侵区域的相关规则方向参数

3.8.9 MI_VgResult_t

➤ 说明

Vg 检测结果对应的结构。

➤ 定义

```
typedef struct _MI_VgResult_t
{
    int32_t alarm[4];
    int32_t alarm_cnt;
    MI\_VgBoundingBox\_t bounding_box[20];
} MI_VgResult_t;
```

➤ 成员

成员名称	描述
alarm[4]	Vg 的报警结果
alarm_cnt	Vg 的报警次数
bounding_box	Vg 触发警报的对象信息

3.8.10 MI_VgBoundingBox_t

➤ 说明

Vg 检测结果对应的物件大小结构。

➤ 定义

```
typedef struct _MI_VgBoundingBox_t
{
    int32_t up;
    int32_t down;
    int32_t left;
    int32_t right;
} MI_VgBoundingBox_t;
```

➤ 成员

成员名称	描述
up	对象的最小 y 坐标信息
down	对象的最大 y 坐标信息
left	对象的最小 x 坐标信息

right	对象的最大 x 坐标信息
-------	--------------

3.8.11 MI_VgDetectThd

- 说明
Vg 建立背景资讯的参数设置的结构。

- 定义

```
typedef struct _MI_VdDetectThd_t
{
    uint8_t function_switch;
    uint8_t detect_thd;
} MI_VgDetectThd;
```

- 成员

成员名称	描述
function_switch	自定义阈值建立背景的功能状态 (0-关, 1-开)
detect_thd	自定义阈值, 范围: [1-50]

4. 错误码

表 4-1 MI_VDF API Error Codes

错误代码	宏定义	描述
0xA0012001	MI_ERR_VDF_INVALID_DEVID	设备 ID 超出合法范围
0xA0012002	MI_ERR_VDF_INVALID_CHNID	通道组号错误或无效区域句柄
0xA0012003	MI_ERR_VDF_ILLEGAL_PARAM	参数超出合法范围
0xA0012004	MI_ERR_VDF_EXIST	重复创建已存在的设备、通道或资源
0xA0012005	MI_ERR_VDF_UNEXIST	试图使用或者销毁不存在的设备、通道或者资源
0xA0012006	MI_ERR_VDF_NULL_PTR	函数参数中有空指针
0xA0012007	MI_ERR_VDF_NOT_CONFIG	模块没有配置
0xA0012008	MI_ERR_VDF_NOT_SUPPORT	不支持的参数或者功能
0xA0012009	MI_ERR_VDF_NOT_PERM	该操作不允许，如试图修改静态配置参数
0xA001200C	MI_ERR_VDF_NOMEM	分配内存失败，如系统内存不足
0xA001200D	MI_ERR_VDF_NOBUF	分配缓存失败，如申请的数据缓冲区太大
0xA001200E	MI_ERR_VDF_BUF_EMPTY	缓冲区中无数据
0xA001200F	MI_ERR_VDF_BUF_FULL	缓冲区中数据满
0xA0012010	MI_ERR_VDF_NOTREADY	系统没有初始化或没有加载相应模块
0xA0012011	MI_ERR_VDF_BADADDR	地址非法
0xA0012012	MI_ERR_VDF_BUSY	系统忙
0xA0012013	MI_ERR_VDF_CHN_NOT_STARTED	通道没有开始
0xA0012014	MI_ERR_VDF_CHN_NOT_STOPED	通道没有停止
0xA0012015	MI_ERR_VDF_NOT_INIT	模块没有初始化
0xA0012016	MI_ERR_VDF_INITED	模块已经初始化
0xA0012017	MI_ERR_VDF_NOT_ENABLE	通道或端口没有 ENABLE
0xA0012018	MI_ERR_VDF_NOT_DISABLE	通道或端口没有 DISABLE
0xA0012019	MI_ERR_VDF_TIMEOUT	超时
0xA001201A	MI_ERR_VDF_DEV_NOT_STARTED	设备没有开始
0xA001201B	MI_ERR_VDF_DEV_NOT_STOPED	设备没有停止
0xA001201C	MI_ERR_VDF_CHN_NO_CONTENT	通道没有资料
0xA001201D	MI_ERR_VDF_NOVASAPCE	映射虚拟地址失败
0xA001201E	MI_ERR_VDF_NOITEM	没有 record 记录

错误代码	宏定义	描述
0xA001201F	MI_ERR_VDF_FAILED	未明确定义的错误

5. PROCFS INTRODUCTION

5.1. cat

- 调试信息

```
# cat /proc/mi_modules/mi_shadow/mi_vdf0
```

- 调试信息分析

记录当前 vdf 的状况以及相关属性、可以动态地获取到这些信息，方便调试和测试。

- 参数说明

```
/mnt # cat /proc/mi_modules/mi_shadow/mi_vdf0

-----Common info for mi_vdf-----
ChnNum      8  EnChnNum    3  PassNum     1  InPortNum   1  OutPortNum  1  CollectSize 0

-----CMDQ kickoff counter-----
DevId       0  current_buf_size 0  Peak_buf_size 0

each dev buf info:
offset      length      used_flag      task_name

-----Common info formi_vdf only dump enabled chn-----
ChnId       0  PassNum     0  EnInPNUM    1  EnOutPNUM   1  MMAHeapName (null)
ChnId       1  PassNum     0  EnInPNUM    1  EnOutPNUM   1  MMAHeapName (null)
ChnId       2  PassNum     0  EnInPNUM    1  EnOutPNUM   1  MMAHeapName (null)
ChnId       0  current_buf_size 0  Peak_buf_size 2000  user_pid    815
each chn buf info:
offset      length      used_flag      task_name
ChnId       1  current_buf_size 0  Peak_buf_size 1000  user_pid    815
each chn buf info:
offset      length      used_flag      task_name
ChnId       2  current_buf_size 0  Peak_buf_size 1000  user_pid    815
each chn buf info:
offset      length      used_flag      task_name

-----chn0 pass0 pipeline delay in us-----
Flow      Min      Max      Avg      Diff
OnPreProcessInputTask  103323  131311  113375  121630
EnqueueInputTask      103738  131592  113675  122407
BarrierInputTask      103747  131828  113723  122416
CheckInputTaskStatus  103759  146785  122698  137559
DequeueInputTask      117069  146799  128701  137567
OnPollingAsyncOutputTaskConfig -1      0      0      0
EnqueueAsyncOutputTask 30      90      70      72
BarrierAsyncOutputTask 33      99      77      79
CheckOutputTaskStatus  66      22731  5922    15176
DequeueOutputTask     11844   22743  14484   15191
FindMADispatch        11864   22768  14504   15217

-----chn1 pass0 pipeline delay in us-----
Flow      Min      Max      Avg      Diff
OnPreProcessInputTask  103506  106854  105941  106456
EnqueueInputTask      103512  107185  106138  106651
BarrierInputTask      103515  107193  106173  106717
CheckInputTaskStatus  103526  141451  117275  127816
DequeueInputTask      122113  141530  126349  127829
OnPollingAsyncOutputTaskConfig -1      0      0      0
EnqueueAsyncOutputTask 29      235    66      50
BarrierAsyncOutputTask 33      2349   151     55
CheckOutputTaskStatus  69      24996  10665   21009
DequeueOutputTask     16218   25007  19742   21020
FindMADispatch        16250   25043  19790   21052

-----chn2 pass0 pipeline delay in us-----
Flow      Min      Max      Avg      Diff
OnPreProcessInputTask  103459  105979  105567  105907
EnqueueInputTask      103475  106608  105931  106407
BarrierInputTask      103480  106805  105969  106416
CheckInputTaskStatus  103489  127798  111808  121576
DequeueInputTask      118757  131181  122129  121584
OnPollingAsyncOutputTaskConfig -1      0      0      0
EnqueueAsyncOutputTask 30      124    54      78
BarrierAsyncOutputTask 34      255    70      87
CheckOutputTaskStatus  82      24899  5904    14763
DequeueOutputTask     12559   24904  16805   14773
FindMADispatch        12607   24939  16990   14813
```

Table 1-1:

参数		描述
Common info for mi_vdf	ChnNum	该 device 的最大 chn 个数
	EnChnNum	该 device enabled 的 chn num 个数
	PassNum	该 device pass 的总数
	InPortNum	该 device inport 的个数
	OutPortNum	该 device outport 的个数
	CollectSize	该 device 对应的 AllocatorCollection 含有 allocator 数目
Common info for mi_vdf only dump enabled chn	ChnId	Chn id
	PassNum	Pass 总数
	EnInPNum	该 chn 的 enabled input port 个数
	EnOutPNum	该 chn 的 enabled output port 个数
	MMAHeapName	如果有 SetChnMMAConf 的话, 该值为对应的 mma heap name; 如果没有设置的话, 该值为 NULL
	current_buf_size	当前申请内存的大小
	Peak_buf_size	使用内存的峰值
	user_pid	当前进程 pid 号

-----Input port common info for mi_vdf only dump enabled port-----										
ChnId	PassId	PortId	user_buf_quota	UsrInjectQ_cnt	BindInQ_cnt	TotalPendingBuf_size	usrLockedInjectCnt			
0	0	0	4	0	0	0	0			
1	0	0	4	0	0	0	0			
2	0	0	4	0	0	0	0			
ChnId	PassId	PortId	newPulseQ_cnt	nextToDoPulseQ_cnt	curWorkingQ_cnt	workingTask_cnt	lazyRewindTask_cnt			
0	0	0	0	0	0	0	0			
1	0	0	0	0	0	0	0			
2	0	0	0	0	0	0	0			
ChnId	PassId	PortId	Enable	bind_module_id	bind_module_name	bind_ChnId	bind_PortId	bind_Type	bind_Param	enable
0	0	0	1	12	mi_divp	2	0	1	0	1
1	0	0	1	12	mi_divp	2	0	1	0	1
2	0	0	1	12	mi_divp	2	0	1	0	1
ChnId	PassId	PortId	SrcFrmrate	DstFrmrate	GetFrame/Ms	FPS	FinishCnt	RewindCnt		
0	0	0	30/ 1	5/ 1	6/1201	4.99	32	0		
1	0	0	30/ 1	5/ 1	6/1200	5.00	27	0		
2	0	0	30/ 1	5/ 1	6/1201	4.99	22	0		

Table 1-2:

参数		描述
Input port bind info (only dump enabled Input port)	ChnId	该 input port 所在的 channel 的 id
	PassId	该 device 的 passid
	PortId	该 device 的 InputPort 的端口 id
	user_buf_quota	该 InputPort 的 buff 数目的 Quota
	UsrInjectQ_cnt	该 InputPort 里的 UsrInjectBufQueue 里的 buff 数目
	BindInQ_cnt	该 InputPort 里的 BindInputBufQueue 里的 buff 数目
	TotalPendingBuf_size	该 InputPort 里的 cur_working_input_queue 里的 buff 的总的 size
	usrLockedInjectCnt	用户层当前拿到了多少个 buf

参数	描述
newPulseQ_cnt	该 InputPort 里的 new_pulse_fifo_inputqueue 里的 buff 数目
nextTodoPulseQ_cnt	该 InputPort 里的 next_todo_pulse_inputqueue 里的 buff 数目
curWorkingQ_cnt	该 InputPort 里的 cur_working_input_queue 里的 buff 数目
workingTask_cnt	该 InputPort 里的 input_working_tasklist 里的 buff 数目
lazyRewindTask_cnt	该 InputPort 里的 lazy_rewind_inputtask_list 里的 buff 数目(需要 retry 的 task 个数)
Enable	1: 使能 0: 失能
bind_module_id	与该 input port 进行了 binded 的 output port 所在的 module 的 id
bind_module_name	与该 input port 进行了 binded 的 output port 所在的 module 的 name (如这里的 mod_name 是 divp) divp->vdf
bind_Chnid	与该 input port 进行了 binded 的 output port 所在的 channel 的 id
bind_PortId	与该 input port 进行了 binded 的 output port 的 id
bind_Type	0x00000001:frame(frame mode, 默认工作方式) 0x00000002:sw_low_latency(低时延工作方式) 0x00000004:realtime mode(硬件直连工作方式) 0x00000008:hw autosync(前后级 handshake, buffer size 与图像分辨率一致) 0x00000010:hw ring(前后级 handshake, ring buffer depth 可以调)
bind_Param	绑定参数
enable	使能
SrcFrmrate	源帧率
DstFrmrate	目标帧率
GetFrame/Ms	实际帧率/所用时间
FPS	实际帧率
FinishCnt	处理完成个数
RewindCnt	需要 Retry 的 task 个数

-----Output port common info for mi_vdf only for enabled port-----									
ChnId	PassId	PortId	usrDepth	BufCntQuota	usrLockedCnt	totalOutPortInUsed	DrvBkRefFifoQ_cnt	DrvBkRefFifoQ_size	
0	0	0	4	6	0	0	0	0	0
1	0	0	4	6	0	0	0	0	0
2	0	0	4	6	0	0	0	0	0
ChnId	PassId	PortId	UsrGetFifoQ_cnt	UsrGetFifoQ_size	UsrGetFifoQ_seqnum	UsrGetFifoQ_discardnum			
0	0	0	0	0	32	0	0	0	0
1	0	0	0	0	27	0	0	0	0
2	0	0	0	0	22	0	0	0	0
ChnId	PassId	PortId	workingTask_cnt	finishedTask_cnt					
0	0	0	0	0					
1	0	0	0	0					
2	0	0	0	0					
ChnId	PassId	PortId	GetFrame/Ms	FPS	FinishCnt	RewindCnt	GetTotalCnt	GetOkCnt	
0	0	0	5/1003	4.98	32	0	32	32	
1	0	0	6/1200	5.00	27	0	27	27	
2	0	0	6/1202	4.99	22	0	22	22	
-----BindPeerInputPortList-----									
ChnId	PassId	PortId	Enable	bind_module_id	bind_module_name	bind_ChnId	bind_PortId	bind_Type	bind_Param enable

Table 1-3:

参数	描述
output port bind info(only dump enabled output port)	ChnId
	该 output port 所在的 channel 的 id
	PassId
	该 device 的 passid
	PortId
	该 device 的 output port 的 id
	usrDepth
	User 可以拿到的 output port buf 的最大个数
	BufCntQuota
	该 OutputPort 的可以申请 buffer 的最大个数
	usrLockedCnt
	从 UsrGetFifoBufQueue 里用户实际拿走了多少个 buffer
	totalOutPortInUsed
	Output 实际申请到的 buffer 个数
	DrvBkRefFifoQ_cnt
	该 OutputPort 的 DrvBkRefFifoQueue 里的 buffer 数目 (For special driver which need to refer back pre-processed buffer)
	DrvBkRefFifoQ_size
	该 OutputPort 的 DrvBkRefFifoQueue 里所占用的 buffer size
	UsrGetFifoQ_cnt
	Dump 到 OutputPort UsrGetFifoBufQueue 所占用 buffer 数目
	UsrGetFifoQ_size
	Dump 到 OutputPort 的 UsrGetFifoBufQueue 所占用 buffer size
	UsrGetFifoQ_seqnum
	OutputPort 拿到的 buffer 总数
	UsrGetFifoQ_discardnum
	该 OutputPort 由于申请新的 buf 需求而丢弃原来的 buffer 累计丢弃数目。
	finishedTask_cnt
	OutputPort 完成 task 的个数
	GetFrame/Ms
	实际帧率/所用时间
	FPS
	实际帧率
	FinishCnt
	已完成个数
	RewindCnt
	Retry task 的个数
	GetTotalCnt
	OutputPort 拿到 buffer 的总数
	GetOkCnt
	统计 E_MI_FRC_OBTAIN 的帧数

```

----- ** Dump all private info for VDF ** -----
VDF Verison: [Jul 13 2021 19:54:08]

----- DEV info for VDF -----
bInited      1      MDEnable      1      ODEnable      1      VGEnable      1      YuvTaskExit  0      MDTaskExit   0      ODTaskExit   0      VGTaskExit   0      DbgLevel     2

----- CHN common info for VDF only dump created chn -----
ChnId      eStatus      phandle      FrameCnt      FrameInv      YImgSize      ImageQCnt
0          2          0xb53061a0    0          0          101376        0
1          2          0xb0a08008    0          0          101376        0
2          2          0xb53079b0    0          0          101376        0

----- Input Port info for VDF only dump created chn -----
ChnId      InGetCnt      InGetFailCnt      InDropCnt      InDoneCnt      InFps
0          32          340          0          32          5.00
1          27          294          0          27          5.00
2          22          242          0          22          4.67

----- Output Port info for VDF only dump created chn -----
ChnId      OutGetCnt      OutGetFailCnt      OutDropCnt      OutDoneCnt      RstGetCnt      RstGetFailCnt      RstDropCnt      RstPutCnt      OutFps
0          32          32          0          32          32          552          0          32          5.00
1          27          28          0          27          27          459          0          27          5.00
2          22          22          0          22          22          373          0          22          5.00

----- MD Attr info for VDF only dump created chn [Ver: 20200210] -----
[====chn:0====]
[c_param]:      Enable      MdBufCnt      VDFIntvl      RstBufSize      SubRstObjLen      SubRstSadLen      SubRstStsLen      cc1_InitArea      cc1_Step
1          4          0          6304          3060          1584          8          2
[s_param]:      width      height      stride      color      mb_size      sad_out_ctr1      roi_md_num      md_alg_mode      pnt[0].x      pnt[0].y      pnt[1].x      pnt[1].y      pnt[2].x      pnt[2].y      pnt[3].x      pnt[3].y
352      288      352      1          1          1          4          1          0          0          351          0          351          287          0          287
[d_param]:      sensitivity      learn_rate      md_thr      obj_num_max
80          2000          16          0

----- OD Attr info for VDF only dump created chn [Ver: 20200108] -----
[====chn:1====]
[c_param]:      Enable      OdBufCnt      VDFIntvl      RstBufSize      nc1rType      alpha      div      roi_od_num      M      MotionSensit
1          4          0          32          1          2          2          4          120          0
[s_param]:      inImgw      inImgH      inImgStride      pnt[0].x      pnt[0].y      pnt[1].x      pnt[1].y      pnt[2].x      pnt[2].y      pnt[3].x      pnt[3].y
352      288      352          0          0          351          0          351          287          0          287
[d_param]:      thd_tamper      tamper_blk_thd      min_duration
3          1          15

----- VG Attr info for VDF only dump created chn [Ver: 0] -----
[====chn:2====]
[c_param]:      Enable      VgBufCnt      VDFIntvl      RstBufSize      obj_size_thd      indoor      func_state      line_number
1          4          0          340          3,00000          1          2          2
[s_param]:      width      height      stride      p[0].x      p[0].y      p[1].x      p[1].y      p[2].x      p[2].y      p[3].x      p[3].y
352      288      352          60, 150          160, 150          110, 190          110, 120          230, 220          330, 220          280, 190          280, 250
line_number      2

```

Table 1-4:

参数	描述
VDF Verison	版本信息
bInited	Vdf 模块是否初始化。（0-否，1-是）
MDEnable	MD 是否使能状态。（0-否，1-是）
ODEnable	OD 是否使能状态。（0-否，1-是）
VGEnable	VG 是否使能状态。（0-否，1-是）
YuvTaskExit	获取 yuv 数据的线程是否退出状态。（0-否，1-是）
MDTaskExit	MD 运算线程是否退出状态。（0-否，1-是）
ODTaskExit	OD 运算线程是否退出状态。（0-否，1-是）
VGTaskExit	VG 运算线程是否退出状态。（0-否，1-是）
DbgLevel	Vdf 模块的 debug 等级。 参数说明： 0- disable 1- error 2- warn 3- info 4- debug 5- trace

参数		描述	
CHN common info for VDF (only dump created chn)	ChnId	Chn id	
	eStatus	0:Destroy 1:created 2:start 3:stop	
	phandle	OD/MD/VG init 句柄指针	
	FrameCnt	帧个数(暂不使用)	
	FrameInv	间隔帧(暂不使用)	
	YImgSize	一张 yImage size(height*width)前级送来的 size	
	ImageQCnt	当前队列里有几张未处理的 yImage 个数	
Input Port info for VDF (only dump created chn)	InGetCnt	Input 端得到的 yImage 次数	
	InGetFailCnt	Input 端得到的 yImage 失败次数	
	InDropCnt	Input 端丢弃的 yImage 次数	
	InDoneCnt	Input 端处理完成的 yImage 数据次数	
	InFps	Input 端获取 yImage 的帧率	
Output Port info for VDF (only dump created chn)	OutGetCnt	Output 端拿到用来存放运算结果的内存次数	
	OutGetFailCnt	Output 端拿到用来存放运算结果的内存失败的次数	
	OutDropCnt	Output 端丢弃的不处理内存次数	
	OutDoneCnt	Output 端写入运算结果到内存成功次数	
	RstGetCnt	用户获取运算结果成功次数	
	RstGetFailCnt	用户获取运算结果失败次数	
	RstDropCnt	丢弃(被新的数据覆盖而丢弃)运算结果的次数	
	RstPutCnt	User 使用完成, 释放运算结果使用内存的次数	
	OutFps	output 运算结果的统计帧率	
MD Attr info for VDF (only dump created chn)	ChnId	通道号	
	[c_param] 常规参数	Enable	当前通道是否使能
		MdBufCnt	设置的 Md 存放结果的缓冲队列深度
		VDFIntvl	暂不使用
		RstBufSize	一次返回结果的大小
		SubRstObjLen	CCL 输出结果的大小
		SubRstSadLen	SAD 输出结果的大小

参数		描述	
		SubRstStsLen	检查结果状态的大小
		ccl_InitArea	CCL 区面积的门槛值
		ccl_Step	CCL 每提高一次门槛值的提升值
	[s_param] 静态参数	width	MD 输入 Image 宽
		height	MD 输入 Image 高
		stride	MD 输入 Image stride
		color	MD 输入 Image 的类型
		mb_size	MD 宏块大小 0- MB_4x4 1- MB_8x8 2- MB_16x16
		sad_out_ctrl	MD 的 SAD 输出格式 0- 16BIT_SAD 1- 8BIT_SAD
		md_alg_mode	CCL 联通区域的运算模式 0- FG 模式 1- SAD 模式
		roi_md_num	侦测区域边界点数量
		Pnt[i].x	侦测区域每个边界点的 x 坐标
		Pnt[i].y	侦测区域每个边界点的 y 坐标
	[d_param] 动态参数	sensitivity	算法灵敏度，范围[10,20,30,.....100]，值越大越灵敏，输入的灵敏
		learn_rate	单位毫秒，范围[1000,30000]，用于控制前端物体停止运动多久时，才作为背景画面
		md_thr	判断移动的门坎值，随不同模式而有不同设定标准
		obj_num_max	CCL 的连通区域数量限制值
OD Attr info for VDF (only dump created chn)	ChnId	通道号	
	[c_param] 常规参数	Enable	当前通道是否使能
		OdBufCnt	设置的 Od 存放结果的缓冲队列深度
		VDFIntvl	暂不使用
		RstBufSize	一次返回结果的大小
	[s_param] 静态参数	inImgW	OD 输入 Image 宽
		inImgH	OD 输入 Image 高
		inImgStride	OD 输入 Image stride
		nClrType	OD 输入影像的类型

参数		描述	
			2- y 数据
		alpha	控制产生参考图像的学习速率
		div	OD 检测窗口的类型 0- 1 个窗口 1- 4 个窗口 2- 9 个窗口
		M	多少张影像更新一次参考图像
		MotionSensit	移动敏感度设置
		roi_od_num	侦测区域边界点数量
		pnt[i].x	侦测区域每个边界点的 x 坐标
		pnt[i].y	侦测区域每个边界点的 y 坐标
	[d_param] 动态参数	thd_tamper	图像差异比例门坎值
		tamper_blk_thd	图像被遮挡区域数量门坎值
		min_duration	图像差异持续时间门坎值
VG Attr info for VDF (only dump created chn)	ChnId	通道号	
	[c_param] 常规参数	Enable	当前通道是否使能
		VgBufCnt	设置的 Vg 存放结果的缓冲队列深度
		VDFIntvl	暂不使用
		RstBufSize	一次返回结果的大小
	[s_param] 静态参数	width	VG 输入 Image 宽
		height	VG 输入 Image 高
		stride	VG 输入 Image stride
		obj_size_thd	滤除物体占感兴趣区域的百分比大小
		indoor	相机架设区域 0- 室外 1- 室内
		func_state	侦测模式 2- VG_VIRTUAL_GATE, 表示模式为虚拟线段。 3- VG_REGION_INVASION, 表示模式区域入侵。
		line_number	虚拟线段的数目, 支持 1 到 4 条虚拟线段 (侦测模式为 VG_VIRTUAL_GATE 起效)
		p[i].x	线段 n 的 x 坐标 (侦测模式为 VG_VIRTUAL_GATE 起效)
		p[i].y	线段 n 的 y 坐标 (侦测模式为 VG_VIRTUAL_GATE 起效)

参数		描述	
		pd[i].x	线段 n 的第一个方向点 (侦测模式为 VG_VIRTUAL_GATE 起效)
		pd[i].y	线段 n 的第二个方向点 (侦测模式为 VG_VIRTUAL_GATE 起效)
		region_dir	设定区域入侵的方向，目前有进入、离开及穿越三种方向可以选择。 0- REGION_ENTER，进入警报区域触发警报。 1- REGION_LEAVING，离开警报区域触发警报。 2- REGION_CROSS，表示穿越警报区域触发警报。 (侦测模式为 VG_REGION_INVASION 起效)
		Pnt[i].x	区域的第 i 个点的 x 坐标 (侦测模式为 VG_REGION_INVASION 起效)
		Pnt[i].y	区域的第 i 个点的 y 坐标 (侦测模式为 VG_REGION_INVASION 起效)

5.2. echo

echo help > /proc/mi_modules/mi_shadow/mi_vdf0

```

/ # echo help > /proc/mi_modules/mi_shadow/mi_vdf0
printk in _MI_SYS_IMPL_Common_WriteProc
-----MI_SYS_IMPL_Common_Help start-----
entry_name=mi_vdf0 eModuleId=0x1 DevId=0x0 InputPortNum=0x1 OutputPortNum=0x1
use cat /proc/mi_modules/mi_vdf/mi_vdf0 to get attr supported by MI_SYS and this module
use echo help > /proc/mi_modules/mi_vdf/mi_vdf0 to get help
use echo dump_buffer parameter1 parameter2 parameter3 parameter4 parameter5 parameter6 > /proc/mi_modules/mi_vdf/mi_vdf0 to
dump buffer by MI_SYS
parameter1 chn_id: means channel id.
parameter2 port_type: value is "iport" or "oport".
parameter3 port_id: means port id of this channel.
parameter4 Queue_name: for input port ,only can be "UsrInject" or "BindInput"; for output port ,only support
"UsrGetFifo".
parameter5 path: result file is stored in which dir, should be absolute path.
parameter6 end_method: when will end dump process. only support "bufnum=xxx", or "time=xxx" (here unit is ms)
or "start/end" pair.
use echo reset_counter [cmdq] [passid] > /proc/mi_modules/mi_vdf/mi_vdf0 to reset debug counters
-----MI_SYS_IMPL_Common_Help end-----
-----** _MI_VDF_OnHelp start ** -----
CMD: help
Sample: echo help > /proc/mi_modules/mi_shadow/mi_vdf0
CMD: log_level
Param1: [0-6] 0-CLOSE 1-FATAL 2-ERROR 3-WARN 4-INFO 5-DEBUG 6-TRACE
Sample: echo log_level 5 > /proc/mi_modules/mi_shadow/mi_vdf0
CMD: sem_check
Param1: [0-1] 0-disable 1-enable
Sample: echo sem_check 1 > /proc/mi_modules/mi_shadow/mi_vdf0
CMD: en_workmode
Param1: [md/od/vg]
Param2: [0-1] 0-disable 1-enable
Sample: echo en_workmode md 1 > /proc/mi_modules/mi_shadow/mi_vdf0

```

```

CMD: en_chn
Param1:[chnId] chnId 0-63
Param2:[0-1] 0-disable 1-enable
Sample: echo en_chn 0 1 > /proc/mi_modules/mi_shadow/mi_vdf0

CMD: auto_check
Param1:[0-1] 0-disable 1-enable
Sample: echo auto_check 1 > /proc/mi_modules/mi_shadow/mi_vdf0

CMD: dump_all
Param1:[0-1] 0-disable 1-enable
Sample: echo dump_all 1 > /proc/mi_modules/mi_shadow/mi_vdf0

CMD: dump_image
Param1:[chnId] chnId 0-63
Param2:[dumpNum] 0-xxxx
Param3:[dumpPath] yourPath
Sample: echo dump_image 0 10 /mnt > /proc/mi_modules/mi_shadow/mi_vdf0

----- ** _MI_VDF_OnHelp end ** -----

```

Echo help 查看可用命令

Table 2-1:

功能	修改打印等级
命令	echo log_level param1> /proc/mi_modules/mi_shadow/mi_vdf0
参数说明	param1 的值是 0~6
举例	echo log_level 5 > mi_vdf0

Table 2-2:

功能	设置 sem_check
命令	echo sem_check param1> /proc/mi_modules/mi_shadow/mi_vdf0
参数说明	param1 的值是 0~1, 0-disable 1-enable
举例	echo sem_check 1 > /proc/mi_modules/mi_shadow/mi_vdf0

Table 2-3:

功能	使能/失能 en_workmode(od/md/vg)
命令	echo en_workmode param1 param2 > /proc/mi_modules/mi_shadow/mi_vdf0
参数说明	param1: [md/od/vg] param2:[0-1] 0-disable 1-enable
举例	echo en_workmode md 0 > /proc/mi_modules/mi_shadow/mi_vdf0

Table 2-4:

功能	开关 vdf chn
命令	echo en_workmode param1 param2 > /proc/mi_modules/mi_shadow/mi_vdf0
参数说明	param1: [chnId] chnId 0-63 param2: [0-1] 0-disable 1-enable
举例	echo en_chn 0 0 > /proc/mi_modules/mi_shadow/mi_vdf0 再次 cat /proc/mi_modules/mi_shadow/mi_vdf0 可以看到 input/output 的 ch0 已经关闭

Table 2-5:

功能	Enable auto_check
命令	echo auto_check param1> /proc/mi_modules/mi_shadow/mi_vdf0
参数说明	param1:[0-1] 0-disable 1-enable

举例	echo auto_check 1 > /proc/mi_modules/mi_shadow/mi_vdf0
----	--

默认关闭，打开后，cat /proc/mi_modules/mi_shadow/mi_vdf0 会多出一些打印帮你检查比如 YImageSize,chn 等等。

Table 2-6:

功能	是否 dump vdf 所有信息
命令	echo dump_all param1> /proc/mi_modules/mi_shadow/mi_vdf0
参数说明	param1:[0-1] 0-disable 1-enable
举例	echo dump_all 1 > /proc/mi_modules/mi_shadow/mi_vdf0

包含 Table 2-5 功能。

Table 2-7:

功能	Dump yuv
命令	echo dump_image [param1, param2, param3] > /proc/mi_modules/mi_shadow/mi_vdf0
参数说明	param1: [chnId] chnId 0-15
	Param2: [dumpNum] 0-xxxx
	Param3: [dumpPath] yourPath
举例	echo dump_image 0 10 /mnt > /proc/mi_modules/mi_shadow/mi_vdf0

例如 dump chn0 10 张 YUV 到/mnt 目录:

```

/mnt # ls
capture          prog_sys
mi.log           share
mi_sys.ko        telnettest.sh
mi_sys_2.ko      vdf[0]_image[1]_3452745.yuv
mi_test.ko       vdf[0]_image[2]_3452551.yuv
prog_cache       vdf[0]_image[3]_3452383.yuv
/mnt # ls -l
vdf[0]_image[4]_3452184.yuv  wifitestapp
vdf[0]_image[5]_3451983.yuv  yuv420sp_640x360.yuv
vdf[0]_image[6]_3451810.yuv  新建文本文档.txt
vdf[0]_image[7]_3451544.yuv
vdf[0]_image[8]_3451348.yuv
vdf[0]_image[9]_3451143.yuv

```